Interakcija čovjeka i robota: analiza poze i gesta

Tara Knežević,
mag. ing. mech.

Zadatak za predaju 2

Prvi dio

Body pose estimation – Z2

- Ova vježba uključuje četiri zadatka:
  - Usporedba ključnih točaka (MediaPipe vs. PennAction) – PDF
  - Izračun kuta zglobova – PDF
  - Izračun nagiba tijela – PDF
  - Live tracking i izračun brzine ruke – TXT datoteka
- Način predaje:
  - Zadatke 1–3 predati u jednom PDF dokumentu
  - Zadatak 4 predati kao jednu .txt datoteku
- Maksimalni broj bodova: **4 boda**

- Predaja se obavlja **tek kada završimo sve vježbe u kolegiju**.

  Ne predajete prije!

Vježba 1

Body pose estimation, MediaPipe

Mediapipe:

- biblioteka za strojno učenje (*Google Research)*
- Koristi se za obradu i analizu medijskih podataka kao što su video, audio i slike u stvarnom vremenu

Google Research

MediaPipe

Mediapipe dependencies:

- Pri radu s MediaPipe bibliotekom, zbog velikog broja ovisnosti koristiti ćemo virtualno okruženje (numpy, scipy, protobuf, jax, matplotlib, opencv...)
- Mediapipe ima gotove wheelove za Python 3.9–3.12
- **Provjereno radi:** Python **3.10 64-bit** + MediaPipe **0.10.x** na Windowsu

Mediapipe modaliteti:

1. **MediaPipe Hands** – detekcija i praćenje šaka
2. **MediaPipe Face Mesh** – detekcija lica i ključnih točaka lica
3. **MediaPipe Pose** – praćenje cjelokupnog tijela i ključnih točaka tijela
4. MediaPipe Objectron – 3D detekcija objekata
5. MediaPipe Holistic - kombinirano praćenje lica, ruku i tijela
6. MediaPipe Solutions – kolekcija unaprijed izgrađenih rješenja i primjera koji pokrivaju razne slučajeve upotrebe

MediaPipe – što sve možemo?

1. Praćenje ruku, lica i tijela – detekcija i praćenje ruku, lica i tijela te njihovih ključnih točaka
2. Segmentacija tijela – prepoznavanje različitih dijelova tijela u slici ili videozapisu
3. Detekcija objekata – identifikacija i praćenje objekata u stvarnom vremenu
4. Označavanje i praćenje kretanja – praćenje kretanja kroz prostor i vrijeme (u videozapisima)

Mediapipe prerequisites:

Instalacija Pythona 3.10 na Windowsu

- https://www.python.org/downloads/ - preuzeti **“Windows installer (64-bit)”** za Python 3.10.x.
- Nakon preuzimanja datoteke pokrenuti instalaciju (označiti **“Add Python 3.10 to PATH”**)
- Provjera nakon instalacije u terminalu: python --version

```
C:\Users\taran>python --version
Python 3.10.11
```

Virtualno okruženje:

- Alat koji omogućava stvaranje izoliranih okruženja za projekte
- venv = *privatni Python sandbox* za svaki projekt

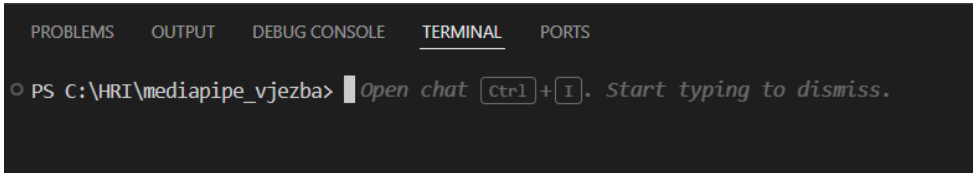
1. Izolacija projekata: Omogućava svakom projektu da koristi vlastite verzije knjižnica, što sprečava konflikte među projektima
2. Kontrola nad ovisnostima: Olakšava upravljanje i održavanje ovisnosti specifičnih za projekt
3. Reproducibilnost: Pomaže u reproduciranju okruženja na različitim strojevima ili među različitim članovima tima

Korištenje virtualnog okruženja:

- Problem: “Could not install... long path” - radite projekt u kratkim putanjama
- Kreiranjem novog direktorija koji sadrži **kopiju Python interpretera** i pip-a, zajedno s vlastitom **kopijom knjižnica** i skripti potrebnih za projekt
- Radni tijek rada s virtualnim okruženjem:
  1. Kreiranje i aktivacija virtualnog okruženja
  2. Instalacija potrebnih knjižnica te rad na projektu
  3. Deaktivacija virtualnog okruženja
  4. Kod ponovnog rada na projektu, reaktivacija virtualnog okruženja

Kreiranje virtualnog okruženja:

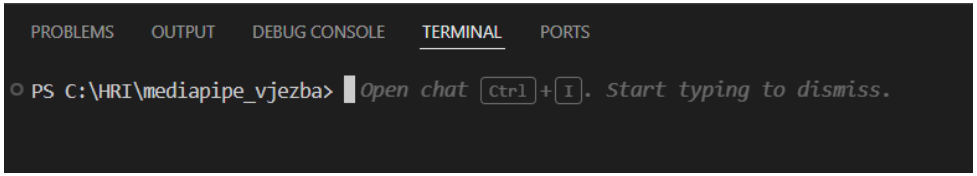
1. Otvorite PowerShell/CMD u folderu projekta:



2. Kreirajte venv: *python -m venv venv*
   - python – pokreće Python interpreter

Kreiranje virtualnog okruženja:

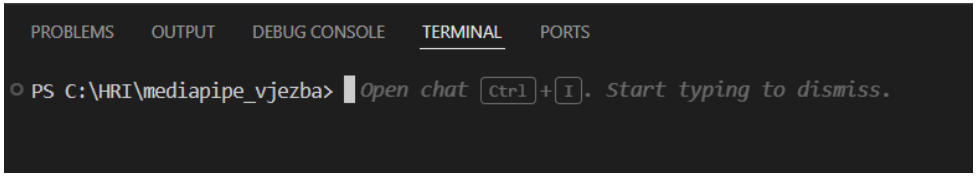
1. Otvorite PowerShell/CMD u folderu projekta:



2. Kreirajte venv: *python -m venv venv*
   - python – pokreće Python interpreter

Kreiranje virtualnog okruženja:

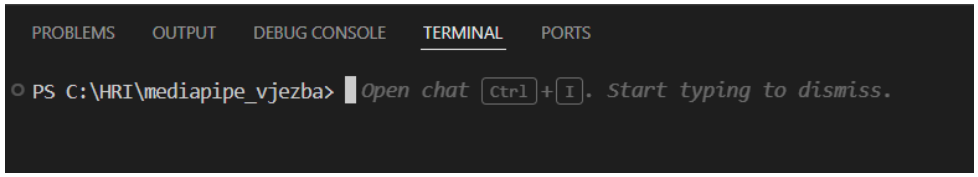
1. Otvorite PowerShell/CMD u folderu projekta:



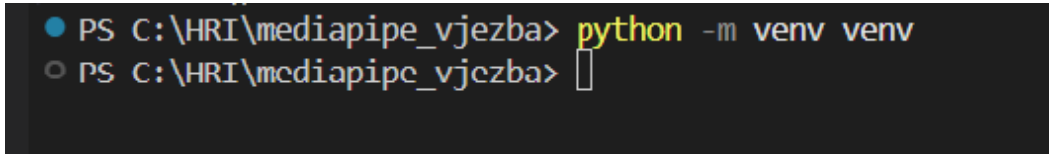
2. Kreirajte venv: *python -m venv venv*
   - python – pokreće Python interpreter
   - -m venv – kaže Pythonu: “napravi virtualno okruženje”
   - venv – ime foldera u kojem će ga napraviti

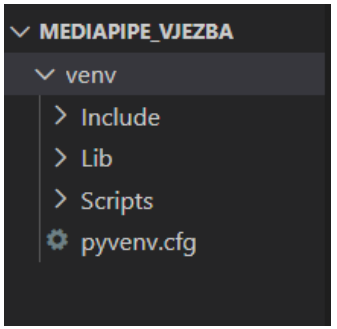
Kreiranje virtualnog okruženja:

1. Otvorite PowerShell/CMD u folderu projekta:



2. Kreirajte venv: *python -m venv venv*
   - python – pokreće Python interpreter
   - -m venv – kaže Pythonu: “napravi virtualno okruženje”
   - venv – ime foldera u kojem će ga napraviti





- Uspješno kreiranje virtualnog okruženja – kreiran je folder ‘venv’

Kreiranje virtualnog okruženja:

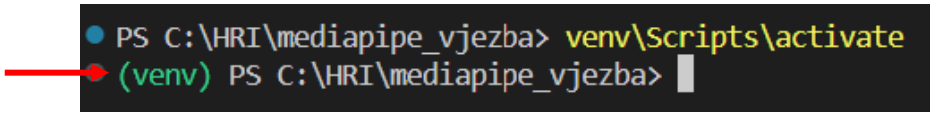
U slučaju da je na računalu instalirano više verzija Pythona

1. Provjera koje verzije postoje: py -0p
2. Kreiranje venv-a s konkretnom verzijom: py -3.10 -m venv venv
3. (Ako py ne radi, doći preko putanje):

   "C:\Users\...\Python310\python.exe" -m venv venv

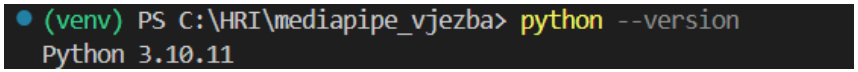
Aktivacija i deaktivacija virtualnog okruženja:

1. Aktivacija (Windows): *venv\Scripts\activate*

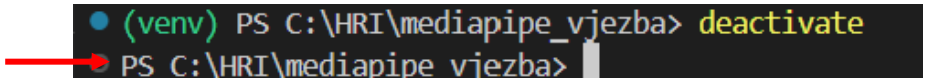


- Uspješna aktivacija virtualnog okruženja – **(venv)** ispred putanje

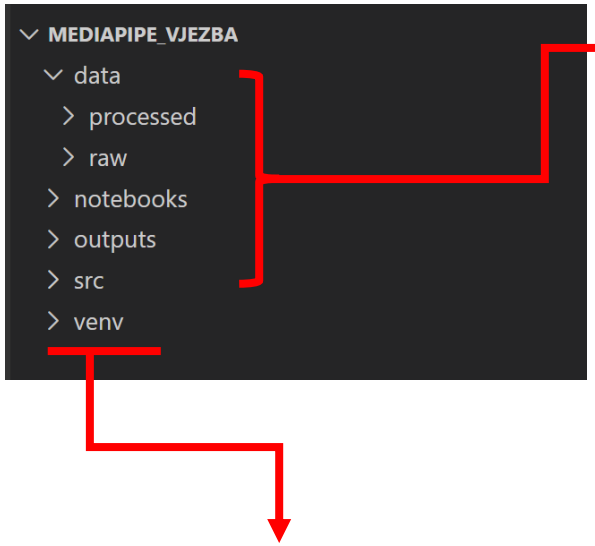
2. Provjera: *python –version*



3. Deaktivacija: *deactivate*



Struktura projekta unutar virtualnog okruženja:



SAMI kreiramo:

1. src - naš Python kod
2. data – podaci
    1. processed – rezultati
    2. raw – ulazni podaci
3. outputs - slike grafova, logovi, modeli, izvještaji
4. notebooks - Jupyter notebooks

Automatski generirano pri izradi virtualnog okruženja

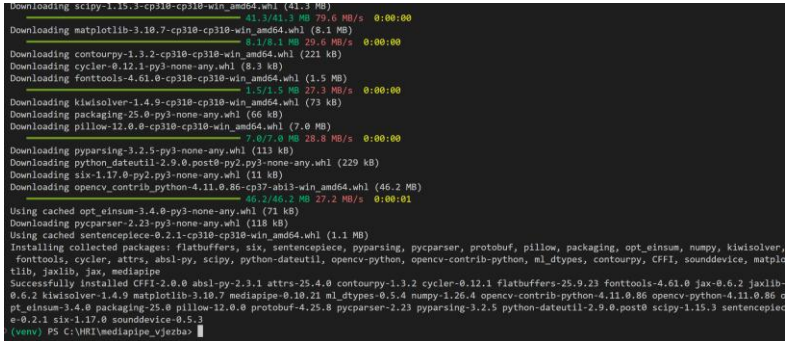
- “venv” ostaje u rootu i kod se nikad ne stavlja u venv

MediaPipe instalacija:

Unutar venv

- Pretpostavka: virtualno okruženje je aktivno

1. pip: python.exe -m pip install --upgrade pip
2. MediaPipe: pip install mediapipe opencv-python
3. Provjera:
   - python
   - import mediapipe as mp
   - print(mp.__version__)




```
(venv) PS C:\HRI\mediapipe_vjezba> python
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import mediapipe as mp
>>> print(mp.__version__)
0.10.21
```

Korištenje virtualnog okruženja:

- Preduvjet: aktivirano virtualno okruženje
- Pozicionirati se u projektni folder unutar terminala
- Pokretanje koda: python src/neki_kod.py

Zadatak 0:

Priprema virtualnog okruženja i instalacija MediaPipe biblioteke

- Prema uputama s prezentacije potrebno je stvoriti vlastito virtualno okruženje
- Prema uputama s prezentacije potrebno je instalirati MediaPipe datoteku

- Provjere:
  - Verzija Pythona
  - Verzija MediaPipe-a

Intel RealSense kamera:

- Dubinska + RGB kamera (dvostruki senzor)
- Daje nam:
  - RGB sliku (kao obična kamera)
  - Depth mapu (udaljenost svakog piksela od kamere)



INTEL REALSENSE



FRAMOS

Intel RealSense instalacija:

1. Instalirati Intel RealSense SDK 2.0 za Windows
   - https://github.com/realsenseai/librealsense/releases/tag/v2.57.4
   - Instalirati **Intel.RealSense.SDK-WIN10-2.57.4.9699.exe** i **Intel.RealSense.Viewer.exe**
   - **NE** u venv - driver je Windows system-level komponenta koja omogućava OS-u da uopće vidi kameru kao uređaj
   - Komponente koje će se instalirati: driver, firmware tools, RealSense Viewer

Intel RealSense instalacija:

2. Instaliraj Python binding u virtualno okruženje
    - **Unutar** aktivnog venv: pip install pyrealsense2
    - Provjera:
        - python
        - import pyrealsense2 as rs
        - print(rs.__file__) - kaže put odakle je modul učitan

Priprema podataka za obradu:

- S e-učenja skinuti mapu „1137.zip” → unzipati → premjestiti mapu u: projekt/data/bpe/raw
- S e-učenja skinuti mapu „src.zip” → unzipati → premjestiti mapu u: projekt/src/bpe

Primjeri vježba1:

- 01_load_image.py - učitavanje i prikaz slike (OpenCV)
- 02_realsense_feed.py – RealSense feed
- 03_find_keypoints.py - MediaPipe Pose + skeleton na slici
- 04_get_keypoints_data.py - čitanje i ispis odabrane ključne točke na 2D slici
- 05_get_keypoints_data_RS.py - čitanje i ispis odabrane ključne točke na RealSense feed-u
- 06_landmarks_list.py – ispis svih ključnih točaka dostupnih na Pose modulu
- 07_angle_calculation.py - funkcija za kut + kut u laktu
- 08_reading_data_pennaction .py - čitanje .mat datoteke i pregled sadržaja
- 09_reading_data_pennaction_xy .py - izvlačenje X,Y za prvi frame

Primjer - Učitavanje slike i prikaz u OpenCV-u:

- OpenCV radi sa slikama kao s matricama piksela (numpy array)
- cv2.imread() učitava sliku iz datoteke
- cv2.imshow() prikazuje sliku u prozoru
- Ovo je temelj za obradu slika bez kamere/real-time-a
- Kod 01_load_image.py
  - Napomena: provjeriti putanju do slike

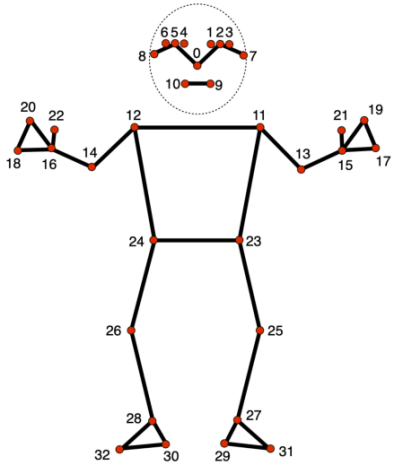
Primjer – RealSense live feed:

- Body pose model mora obrađivati slike frame-by-frame
- LIVE FEED znači:
  - Stalno čitanje nove slike
  - Detekcija
  - Prikaz rezultata

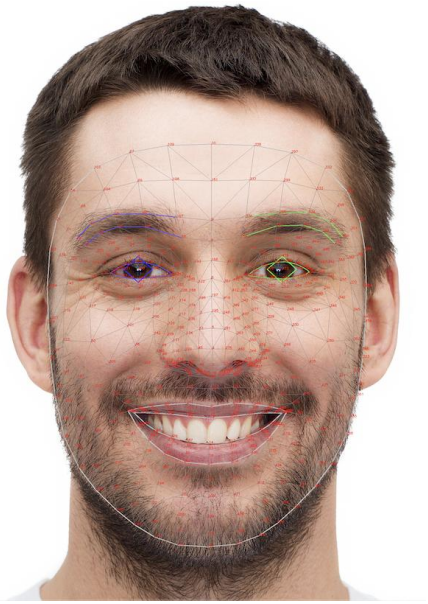


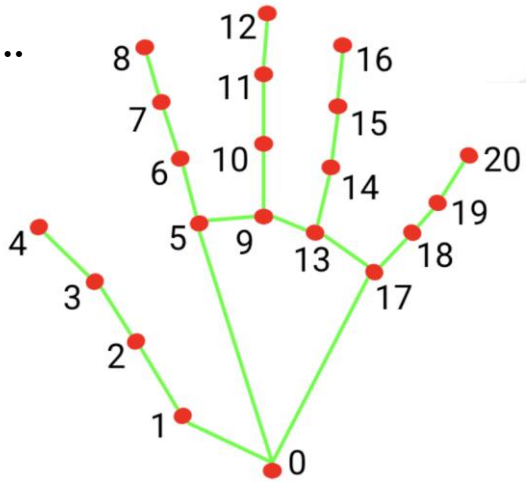
- Dobivamo stream kroz pipeline:
  1. Konfiguriraj senzore
  2. Start
  3. U beskonačnoj petlji čitaj frame
  4. Pretvori u numpy sliku
- Kod 02_realsense_feed.py

MediaPipe moduli:



1. Pose:
   - 33 ključne točke tijela
   - Skeleton: ramena, kukovi, koljena, laktovi…
2. Face Mesh:
   - 468 ključnih točaka lica
   - Detektira lice, oči, usta, obrve…
3. Hands:
   - 21 ključna točka po ruci

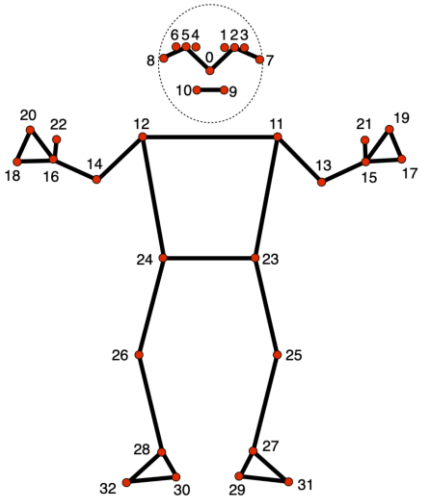


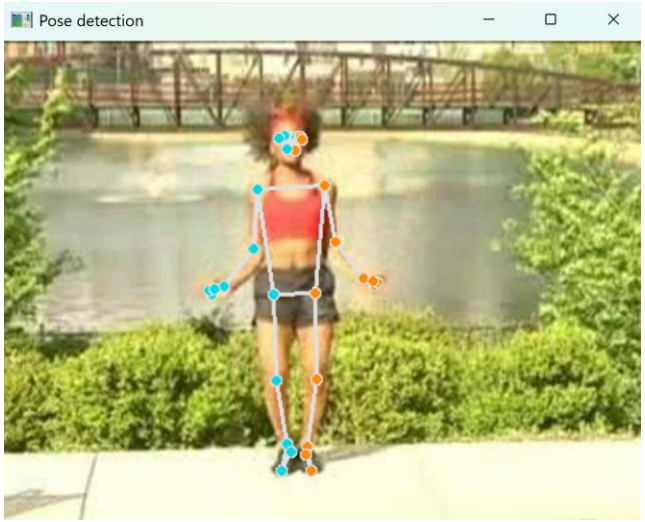


Primjer MediaPipe Pose na slici:



- OpenCV radi s BGR, a MediaPipe s RGB formatom
- Informacije o 33 ključne točke i veze između njih





- Kod 03_find_keypoints.py

Što je MediaPipe landmark?

- MediaPipe landmarks – X, Y, Z koordinate
- results.pose_landmarks.landmark - Kada primijenimo MediaPipe Pose model dobijemo listu od 33 ključne točke tijela
- Svaki landmark sadrži 4 vrijednosti:
  1. **x** – normalizirana horizontalna koordinata (0–1)
  2. **y** – normalizirana vertikalna koordinata (0–1)
  3. **z** – relativna dubina (procjena modela, ne stvarna mjera)
  4. **visibility** – vjerojatnost da je točka vidljiva (0–1)

Primjer čitanje podataka o ključnim točkama:

VRIJEDI ZA 2D SLIKE

- Normalizirane koordinate:
  - Kad MediaPipe vrati landmark.x i landmark.y, oni nisu u pikselima nego u rasponu **od 0 do 1** - Iz RGB slike NEMAMO stvarnu dubinu u metrima
  - Korisno: Slika 640×480 i slika 1920×1080 daju iste x,y vrijednosti

  - landmark.z je **procijenjena** relativna dubina modela (**ne mjerena**) treniranog na 3D podacima
  - **Dubina** - vrijednost kaže je li zglob ispred ili iza ravnine tijela

- Kod 04_get_keypoints_data.py

Primjer čitanje podataka o ključnim točkama:

VRIJEDI ZA PODATKE DOBIVENE IZ KAMERE S DUBINOM

- Vrijednosti x i y – stvarni pikseli
- Vrijednosti z – stvarna dubina [m]

- Kamera ima **intrinzične parametre** (žarišna duljina, optičko središte, geometrija senzora) – RS biblioteka nudi opciju čitanja tih podataka
- Prema poznatim informacijama očitanim u pikselima, možemo izvesti de-projekciju u 3D → vrijednosti **x i y u metrima**
- Nakon deprojekcije dobivamo (X, Y, Z) u [m] u koordinatnom sustavu kamere

- Kod 05_get_keypoints_data_RS.py

Primjer čitanje podataka o ključnim točkama:

- Visibility:
  - Vrijednost u rasponu 0–1
  - Signalizira koliko je model siguran da je landmark stvarno vidljiv (pouzdanost podataka)
  - Visibility < 0.5 → točka je možda skrivena
  - Ruka iza tijela → low visibilityice okrenuto od kamere → neki landmarkovi imaju visibility = 0
  - Kod računanja kuta ili položaja moramo ignorirati landmarkove niske vidljivosti

Primjer pristupanje ključnoj točki u kodu:

- Pristupanje ključnoj točki ‘NOSE’:

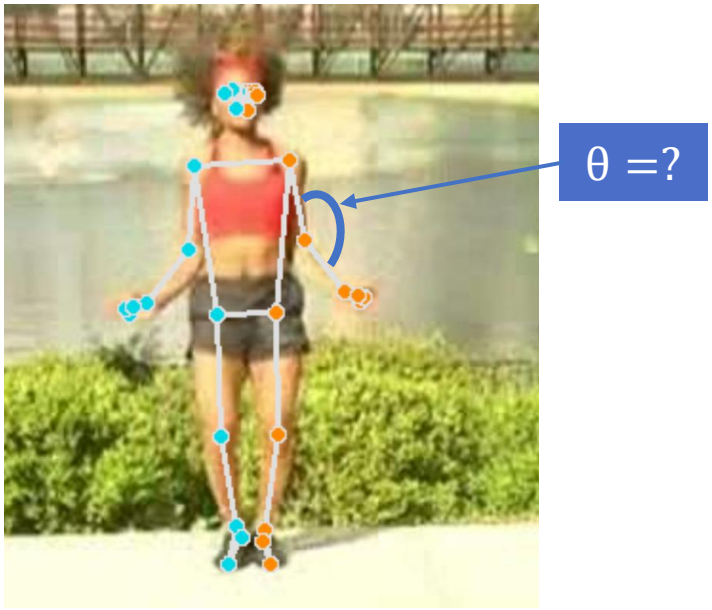
```
lm = results.pose_landmarks.landmark
nose = lm[mp_pose.PoseLandmark.NOSE.value]
```

  - Čitanje vrijednosti:
    - nose.x
    - nose.y
    - nose.z
    - nose.visibility
  - Popis svih landmarkova koje modul ima – kod 06_landmarks_list.py

```
(venv) PS C:\HRI\mediapipe_vjezba> python .\src\06_landmarks_list.py
0 NOSE
1 LEFT_EYE_INNER
2 LEFT_EYE
3 LEFT_EYE_OUTER
4 RIGHT_EYE_INNER
5 RIGHT_EYE
6 RIGHT_EYE_OUTER
7 LEFT_EAR
8 RIGHT_EAR
9 MOUTH_LEFT
10 MOUTH_RIGHT
11 LEFT_SHOULDER
12 RIGHT_SHOULDER
13 LEFT_ELBOW
14 RIGHT_ELBOW
15 LEFT_WRIST
16 RIGHT_WRIST
17 LEFT_PINKY
18 RIGHT_PINKY
19 LEFT_INDEX
20 RIGHT_INDEX
21 LEFT_THUMB
22 RIGHT_THUMB
23 LEFT_HIP
24 RIGHT_HIP
25 LEFT_KNEE
26 RIGHT_KNEE
27 LEFT_ANKLE
28 RIGHT_ANKLE
29 LEFT_HEEL
30 RIGHT_HEEL
31 LEFT_FOOT_INDEX
32 RIGHT_FOOT_INDEX
```

Primjer izračun kuta koristeći ključne točke:

- U analizi ljudskog pokreta **kutovi između segmenata** tijela najčešće su najinformativniji parametar, jer izravno opisuju položaj i kretnju zglobova
- Za izračun kuta između dva segmenta uvijek su potrebne **tri različite ključne točke**
- **Primjer:** izračunati kut u laktu lijeve ruke

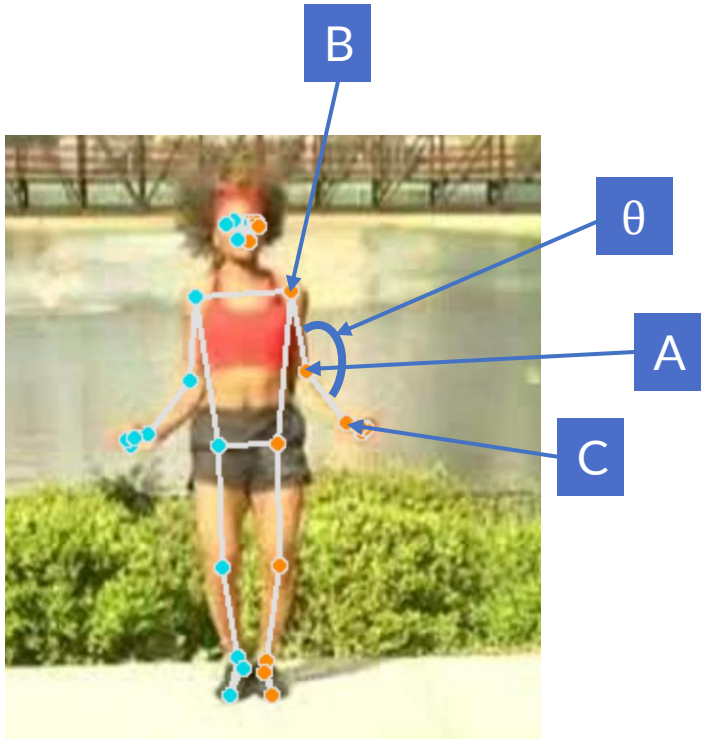


Odabrane ključne točke:

- Lijevo rame - LEFT_SHOULDER
- Lijevi lakat - LEFT_ELBOW
- Lijevo zapešće - LEFT_WRIST

Primjer izračun kuta koristeći ključne točke:

Ovo je ista formula koju koristimo u funkciji calculate_angle(a, b, c) u kodu



- Određivanje vektora:

$$\overrightarrow{AB} = B - A = (x_B - x_A, y_B - y_A)$$

$$\overrightarrow{AC} = C - A = (x_C - x_A, y_C - y_A)$$

- Matematička norma:

$$\left\|\overrightarrow{AB}\right\| = \sqrt{{x_{AB}}^2 + {y_{AB}}^2}$$

$$\left\|\overrightarrow{AC}\right\| = \sqrt{{x_{AC}}^2 + {y_{AC}}^2}$$

- Skalarni produkt:

$$\overrightarrow{AB} * \overrightarrow{AC} = x_{AB} * x_{AC} + y_{AB} * y_{AC}$$

- Kut između dva vektora:

$$\cos\theta = \frac{\overrightarrow{AB} * \overrightarrow{AC}}{\left\|\overrightarrow{AB}\right\| * \left\|\overrightarrow{AC}\right\|}$$

- Kod 07_angle_calculation.py

Javno dostupni setovi s temeljnom istinom (engl. ground truth):

- Skupovi podataka koji su dostupni za istraživače, developere i entuzijaste, besplatni te se slike i snimke smiju koristiti (GDPR)
- Omogućuju testiranje, treniranje modela i razvoj aplikacija bez potrebe za prikupljanjem vlastitih podataka
- Načini prikupljanja informacija:
  1. Optički sustavi - kamere snimaju reflektivne markere postavljene na tijelu subjekta
  2. Inercijalni senzori -nuređaji s akcelerometrima i žiroskopima za praćenje pokreta
  3. Dubinske kamere - kamere koje snimaju trodimenzionalne podatke o pokretima tijela
  4. Manualno označavanje - ručno označavanje ključnih točaka na tijelu u video zapisima

PennAction dataset:

- https://dreamdragon.github.io/PennAction/
- Sadrži 2326 video sekvenci s 15 različitih radnji i ground truth o ljudskim zglobovima za svaku sekvencu

```
/frames  ( all image sequences )
   /0001
      000001.jpg
      000002.jpg
      ...
   /0002
    ...
/labels  ( all annotations )
    0001.mat
    0002.mat
    ...
/tools   ( visualization scripts )
    visualize.m
    ...
```

Organizacija podataka

```
annotation =

      action: 'tennis_serve'
        pose: 'back'
           x: [46x13 double]
           y: [46x13 double]
  visibility: [46x13 logical]
       train: 1
        bbox: [46x4 double]
  dimensions: [272 481 46]
     nframes: 46
```

Podaci koji se mogu očitati

```
1.  head
2.  left_shoulder   3.  right_shoulder
4.  left_elbow      5.  right_elbow
6.  left_wrist      7.  right_wrist
8.  left_hip        9.  right_hip
10. left_knee       11. right_knee
12. left_ankle      13. right_ankle
```

Ključne točke o kojima postoje informacije

Primjer čitanje podataka PennAction:

- Čitamo strukturu dataseta kako bismo znali gdje se nalaze koordinate i kako su organizirane po frameovima
- Otvaramo annotation label datoteku u .mat formatu
- Ispisujemo što dataset sadrži
- Cilj nije odmah obrađivati, nego pogledati strukturu

- x → koordinate svih jointova kroz sve frameove
- y → koordinate svih jointova kroz sve frameove
- visibility → koji joint je vidljiv (1 ili 0)
- bbox → bounding box osobe
- nframes → broj frameova
- action → naziv radnje (npr. pullup)

- Kod 08_reading_data_pennaction.py

Primjer čitanje podataka PennAction – x,y:

- Cilj je izdvojiti koordinate iz dataseta za **određeni frame**
- Dobiti Python listu [x1, x2, ...] i [y1, y2, ...] - ground truth

```
1.  head
2.  left_shoulder   3.  right_shoulder
4.  left_elbow      5.  right_elbow
6.  left_wrist      7.  right_wrist
8.  left_hip        9.  right_hip
10. left_knee       11. right_knee
12. left_ankle      13. right_ankle
```

Ključne točke o kojima postoje informacije

- U PennAction-u **x i y su koordinate u pikselima** (nisu normalizirane)

- Kod 09_reading_data_pennaction_xy.py

Samostalni zadatak 1:

Usporedba ground truth i MediaPipe

- Odabrati 5 fotografija iz mape podaci
- Za svaku fotografiju učitati ground truth iz PennAction dataseta
- Za iste ključne točke dobiti poziciju pomoću MediaPipe Pose
- Rezultate usporediti i prikazati u tablici

ID slike	Ground truth (x, y)	MediaPipe (x, y)	Razlika

Samostalni zadatak 2:

Izračun kuta zgloba

- Odabrati 5 fotografija iz mape podaci
- Izračunati kut:
  - desni lakat
  - lijevo koljeno
- Ispisati rezultate u terminal i prikazati tablicu

ID slike	Lakat (°)	Koljeno (°)

Samostalni zadatak 3:

Nagib tijela

- Odabrati 5 fotografija iz mape podaci
- Sami odabrati ključne točke torza
- Izračunati nagib u odnosu na vertikalu
- Rezultat prikazati u tablici

ID slike	Nagib (°)

Samostalni zadatak 4:

Live tracking i brzina ruke

- Koristiti live feed iz kamere
- Pratiti poziciju zapešća u vremenu
- Izračunati brzinu gibanja u m/s
- Spremiti rezultate u .txt datoteku